

Civil Craft NPC Phase Authoring Guide

For the story team and the team members implementing dialogue in Unity. Updated 7 September 2026.

This guide explains how to turn a story beat into an NPC phase using the systems already in Civil Craft. Every phase needs a clear starting condition, a player experience, and an ending condition. Writers specify those decisions; the Unity implementer assigns scene objects, assets, and events.

A phase does not require a contract or a tutorial. An NPC can simply talk, travel, introduce a material, or unlock a feature.

Contents

1. How phases work
2. Setting up a phase
3. Writing dialogue and material sequences
4. Offering and completing contracts
5. Events and available actions
6. Cinematics and other presentations
7. Feature unlocks
8. Movement and idle behavior
9. Saving and replay behavior
10. Worked example
11. Writer handoff template
12. Testing and troubleshooting

How phases work

An NPC phase is one stage in that NPC's story. It identifies where the NPC should wait, what happens when the player interacts, and which actions lead to the next stage.

An example sequence is:

Greeting → Introduce materials → Offer a bridge contract → Turn in the contract → Travel to town

The NPC Progression Manager controls one NPC's ordered phase list. The NPC Contract Giver supplies the contract interaction. A Target Location is a scene Transform used as a destination, not a build location asset.

A conversation ending does not automatically move the NPC to the next phase. For a contract-free phase, wire an event that explicitly advances it. Contract phases can advance automatically when their contract is recorded as completed.

Setting up a phase

1. Select the NPC in the scene Hierarchy.
2. Find its **NPC Progression Manager** component.
3. Expand **Phases** and add an entry.
4. Give it a descriptive **Phase ID**, such as `Bhan_IntroduceMaterials`.
5. Assign a **Target Location** where the NPC should stand.
6. Choose ordinary dialogue, an ordered dialogue/material sequence, or a contract.
7. Clear unintended unlock fields and configure repetition.
8. Wire the event that ends the phase, unless contract completion handles advancement.
9. Test the phase and the transition into the following phase.

Main Inspector fields

Field	Purpose	Authoring guidance
Phase ID	Stable story-stage identifier	Use a unique name within the NPC sequence. Keep published IDs stable.
Target Location	NPC destination and waiting position	Assign a stationary scene marker, even for a conversation at the current location.
Travel Waypoints	Ordered intermediate travel markers	Used by waypoint movement; plan safe routes over ground and bridges.
Contract	Contract offered in this phase	Leave empty for ordinary story conversations.
Target Build Location	Site associated with the contract	Assign the correct Build Location component.
Linked Cargo	Optional cargo tied to the contract	Its weight follows the contract's live load.
Dialogue Prompt	Contract-free interaction prompt	Write a short action such as "Talk to Professor Bhan."
Play Dialogue On Arrival	Starts optional dialogue when the NPC arrives	Otherwise, the player starts it by interacting. Use cautiously on contract phases.
Repeat Dialogue	Allows replay of optional dialogue	Disable for a one-time scene-visit conversation. See saving rules below.
Enable Idle Roaming	Enables local wandering in this phase	Use for waiting phases away from cliff edges.

Keep destination markers in a stationary group such as **NPC_Phase_Targets**. Do not parent them to the NPC they are guiding; they would move with that NPC.

Writing dialogue and material sequences

A simple conversation without a contract

Leave **Contract** empty. Populate **Phase Dialogue**, including the speaker name and sentences. Set **Dialogue Prompt**, then choose whether the player interacts or the NPC speaks on arrival.

Example dialogue:

Professor Bhan: "Before we start, you should understand what your bridge carries."

Professor Bhan: "The road deck supports vehicles. Other structural members help support the deck."

Use **On Dialogue Finished** to start the next story action. A tutorial is not needed just to make the NPC speak.

Mixing conversation with material introductions

Use **Optional Dialogue Sequence** to arrange Dialogue and Material Unlock entries in a specific order.

Step	Type	Content
1	Dialogue	Introduce the road deck.
2	Material Unlock	Assign the Wood Road material SO.
3	Dialogue	Explain why supports are needed.
4	Material Unlock	Assign the Beam material SO.
5	Dialogue	Tell the player what happens next.

The player closes each dialogue or material panel before the next sequence entry starts. A material introduction records the acknowledged material in the Almanac. It does not replace the contract's allowed-material settings or give an unlimited inventory of pieces.

Use **Material Button Label** to override a particular material entry's button. Otherwise, the phase's **Material Introduction Button Label** is used, normally **GOT IT**.

When Optional Dialogue Sequence has usable entries, it replaces the older Phase Dialogue and Material Introduction flow. Do not fill both expecting both to play.

Older material introduction fields

Existing phases can use **Material Introduction** plus **Material Introductions**. These show the single assigned material first, followed by additional materials. **On Material Introduction Closed** fires after the final panel is dismissed.

In this older flow, **On Dialogue Finished** can fire while material panels are still queued or open. Put actions that must wait for the last material panel on **On Material Introduction Closed**, or use the ordered sequence.

Offering and completing contracts

Assign the Contract SO and Target Build Location to the phase. Write the contract conversation on the **Contract SO**.

Dialogue field	When it is used
Offer Dialogue	Before the player reviews the contract offer.
Continue Offer Dialogue	After the player presses Accept Contract.
Reminder Dialogue	When the player returns before finishing the task.
Finished Contract Dialogue	In the contract completion or turn-in flow.

The normal offer flow is:

Interact → Offer dialogue → Contract details → Accept → Continue offer dialogue → Post-offer actions

If the player cancels the offer, they can return later. Do not design essential acceptance rewards around the initial interaction event, which happens before acceptance.

Contract phases normally use the giver's offer, reminder, and completion dialogue instead of ordinary Phase Dialogue on interaction. An **Optional Dialogue Sequence** can run after acceptance. Avoid using **Play Dialogue On Arrival** to introduce a second dialogue flow while offering a contract.

What advances a contract phase

With **Automatically Advance On Contract Completion** enabled, the progression manager listens for completion of its current phase's contract and advances one phase.

For contracts requiring a return to the giver, saving a successful bridge prepares the task for turn-in; it is not the same event as completing the turn-in. **Auto Collect Reward** contracts can complete during finalization instead.

Do not also wire a second advance action for the same completion unless that behavior is intentional. Never advance the offer phase merely because the offer dialogue closed if the NPC still needs to wait for the player to build and turn in that contract.

Events and available actions

Events are the Inspector lists with a + button. They connect a moment in the phase to a method on a scene component.

Event timing

Event	Timing	Typical use
On NPC Interacted	When the player interacts with this NPC	Start an interaction-specific action.

Event	Timing	Typical use
On NPC Arrived	After movement arrives and the destination phase activates	Reveal a scene object or begin a presentation.
On Dialogue Finished	After legacy phase dialogue, or after all steps of the ordered sequence	Advance the story after conversation.
On Material Introduction Closed	After the material presentation finishes	Continue after the player acknowledges materials.
On Progression Finished	When advancing beyond the last phase	Hand off to another sequence.
On Movement Failed	When the movement system reports failure	Trigger an approved recovery or debugging response.

In an ordered sequence, the material-closed event runs before the dialogue-finished event at the end if a material was displayed. Assign phase advancement to only one of them.

Invoke Interaction Event Only Once limits the interaction event during the scene visit. It does not by itself control dialogue repetition; use Repeat Dialogue for that.

Actions already available in the project

Desired action	Component and method	What to assign
Advance one phase	<code>NPCProgressionManager.AdvanceToNextContract()</code>	The NPC's progression component. Works for contract-free phases too.
Travel to a specific phase	<code>NPCProgressionManager.MoveToPhase(int)</code>	The destination phase index.
Play a cinematic	<code>CinematicDirector.PlayCinematic()</code>	The configured cinematic scene object.
Start a tutorial	<code>TutorialManager.PlayTutorial(sequence)</code>	A TutorialSequence asset.
Queue a tutorial	<code>TutorialManager.QueueTutorial(sequence)</code>	A TutorialSequence asset.
Show a lesson	<code>LessonTrigger.ShowLesson()</code>	A configured LessonTrigger with a LessonData asset.
Play a separate conversation	<code>DialogueTrigger.TriggerDialogue()</code>	A configured DialogueTrigger.
Show or hide an object	<code>GameObject.SetActive(bool)</code>	The target GameObject and desired active state.

An event can call methods compatible with Unity's event picker. It cannot interpret a sentence such as “open the gate.” That action needs an existing component method or a request to the programming team.

Wiring movement after a conversation

1. Expand **On Dialogue Finished** on the source phase.
2. Press **+** and drag the NPC into the object field.
3. Select **NPCProgressionManager** → **MoveToPhase(int)**.
4. Enter the destination index.

Indices start at **0**. The first phase is 0, the second is 1, and the third is 2. Reordering phases requires rechecking these integer assignments. Alternatively, use **AdvanceToNextContract** when the intended destination is always the next entry.

Cinematics and other presentations

Contract introduction cinematics

Assign **Cinematic After Offer** on the phase that actually offers the contract. It runs after acceptance and the follow-up offer dialogue. Different contract phases can reference different cinematics.

This field uses the explicit contract-offer playback path, which can replay an accepted-offer presentation even when normal startup playback is one-shot.

Cinematics without a contract

Wire **On Dialogue Finished** → **CinematicDirector.PlayCinematic()**. Configure the cinematic's camera, shots, and actors. Normal **PlayCinematic** respects **Play Only Once** and its saved cinematic ID.

If the NPC must move after the cinematic, connect that action to **On Cinematic Finished** on the cinematic, rather than starting movement alongside the cinematic.

Avoid overlapping interfaces

Event-list order is not a waiting system. Starting a cinematic in the first entry and a lesson in the second can open both together. The current contract-offer handler can also start its cinematic and optional dialogue sequence in the same handler.

For a deliberate presentation order, chain completion callbacks or separate the story into phases. Ask the programming team for a completion hook if the required one is unavailable.

A navigational instruction such as “Go to the workbench” should use the existing guide flow. Do not start a blocking tutorial merely to show a direction prompt.

Feature unlocks

A contract-free optional conversation can permanently unlock a feature after finishing.

1. Set **Unlock Feature ID After Dialogue** to the feature's supported ID, for example `minimap`.
2. Set **Feature Unlock Display Name** and optional **Feature Unlock Icon**.
3. Enable **Show Feature Collect Popup** if the player should press Collect first.
4. Test with a save that does not already own the feature.

New phases currently default to the minimap feature ID. Clear that field on phases that should unlock nothing. A display name alone does not define a new feature; custom IDs also need a system that listens to and uses them.

When the popup is available and enabled, the feature is granted after Collect. If the popup system is unavailable, the current implementation can grant the feature directly.

On Dialogue Finished does not wait for the feature Collect button. Do not put an action there that assumes the feature has already been collected. Feature unlocking through these phase dialogue fields is reserved for contract-free phases; contract rewards have their own feature reward configuration.

Movement and idle behavior

Movement mode is selected on the NPC Progression Manager, not independently on every phase.

NavMesh mode uses navigation data and compatible links. **Waypoints mode** supports walking over ground and bridge colliders without requiring a complete NavMesh path; its configuration can also try compatible scene links first.

For the story team, supply the destination and intended route. The Unity implementer should verify ground layers, bridge availability, links, and collision geometry. A scene marker on the far side of an unbuilt bridge does not create a safe crossing.

Enable Idle Roaming is phase-specific. The roaming radius, speed, and pause settings are shared on the NPC manager. Enable it for natural waiting behavior; keep the area clear and away from edges.

Interactions are locked while the NPC travels. The manager also has movement-failure and optional warp-recovery settings. Recovery is a safeguard, not a substitute for a working route.

Saving and replay behavior

Use a unique **Progression Save ID** for each NPC sequence. Enable **Persist Progression** when the NPC should resume its story state after loading. Phase IDs and contract IDs should remain stable after release.

The current save records the phase and whether travel was in progress. Resume behavior depends on movement mode and the saved state. Do not promise that every transition animation replays exactly after loading.

State	Scope
NPC phase	Saved when progression persistence is enabled.

State	Scope
Feature ownership	Saved through the feature unlock system.
Material discovery	Recorded in player data when acknowledged.
Repeat Dialogue suppression	Scene visit, not a permanent conversation history.
Invoke Interaction Event Only Once	Scene visit.
Introduce Material Only Once	Scene visit.
Cinematic Play Only Once	Saved for normal cinematic playback; explicit accepted-offer playback differs.

Normal restoration activates the resolved phase without firing its arrival event or arrival dialogue again. Essential restored state must not depend only on **On NPC Arrived**.

Treat future additions to published sequences carefully. Stable phase IDs help restoration, but explicit MoveToPhase integer references still need manual review after reordering.

Worked example

This is a suggested authoring pattern, not a replacement for the current scene's story.

Index	Phase ID	Player experience	Exit condition
0	Bhan_Greeting	Player talks to Bhan. No contract or unlock.	On Dialogue Finished calls AdvanceToNextContract.
1	Bhan_IntroduceMaterials	Bhan arrives at the cart. Dialogue and material panels run in order.	Final On Dialogue Finished advances once.
2	Bhan_OfferFirstBridge	Bhan offers the assigned contract at the workbench.	Actual contract completion advances automatically.
3	Bhan_TownDebrief	Bhan travels to town and waits for a conversation.	On Dialogue Finished advances past the last phase and fires On Progression Finished.

In phase 1, use Dialogue → Wood Road material → Dialogue → Beam material → closing dialogue. Clear the default feature unlock if this phase should not grant the minimap.

In phase 2, keep optional post-offer dialogue empty if Cinematic After Offer is assigned, unless the intended sequencing has been explicitly wired. The NPC should remain in this phase while the player fulfills the contract.

Writer handoff template

Copy this section for every new story phase. Replace each bracketed instruction before handing it to the Unity implementer.

Phase specification

- **NPC and scene:** [Character and scene name]
- **Phase ID:** [Stable descriptive ID]
- **Story purpose:** [What the player should learn or understand]
- **NPC destination:** [Named marker and intended standing direction]
- **Route requirements:** [Waypoints, required completed bridge, or other conditions]
- **Start condition:** [Interaction, arrival, or another explicitly wired event]
- **Interaction prompt:** [Short player-facing text]
- **Dialogue and material sequence:** [Speaker, exact sentences, and material assets in order]
- **Contract and build site:** [Assigned assets or None]
- **Cinematic or lesson:** [Assigned presentation and exactly when it starts]
- **Feature unlock:** [Supported ID, display name, icon, and whether Collect is required; otherwise None]
- **Idle roaming:** [Enabled or disabled]
- **Repeat behavior:** [Whether dialogue and interaction events repeat]
- **Exit condition:** [Exact event or contract completion that advances the phase]
- **Next phase:** [Phase ID and its current list index]
- **Reload expectation:** [What should be visible and interactable when loading here]

Dialogue writing notes

Keep sentences short enough to read comfortably on a phone. Give the player the objective before explaining optional background. Introduce a material immediately before its panel so the connection is clear. Write reminder dialogue that still makes sense after the player has attempted the bridge several times.

Testing and troubleshooting

Before approving a sequence

- [] All destination markers and required assets are assigned.
- [] Progression Save ID is unique and phase IDs are descriptive.
- [] Default minimap unlock is cleared where it is not intended.
- [] Every contract-free phase has a deliberate exit condition.
- [] A single transition is not wired to advance twice.
- [] The NPC reaches the destination without falling or relying on recovery warps.
- [] Dialogue and material panels close before the next required action.
- [] Cancelling and later accepting a contract both work.

- [] Reminder and turn-in dialogue use the correct contract.
- [] The guide remains a guide and does not block building as a tutorial.
- [] Mobile dialogue fits and prompts are easy to read and tap.
- [] Returning after a scene reload restores a usable phase.
- [] Tests cover already-collected features and materials as well as a fresh save.

Common symptoms

Symptom	Check first
NPC does nothing after talking	Is the contract-free phase's exit event wired?
NPC starts moving too soon	Is advancement connected to interaction or legacy dialogue completion instead of the final presentation callback?
Two panels open together	Are independent asynchronous actions attached to the same event?
Wrong dialogue plays	Is this a contract phase using Contract SO dialogue, or does an ordered sequence override Phase Dialogue?
Minimap unlock appears unexpectedly	Check the default Unlock Feature ID After Dialogue value.
Cinematic does not replay	Check its ID and Play Only Once setting, and which playback method is called.
NPC cannot cross the bridge	Check movement mode, actual bridge colliders, route markers, and compatible navigation links.
Scene reload skips an arrival action	Normal restored-phase activation does not invoke arrival events.
A reordered story goes to the wrong phase	Recheck every MoveToPhase integer.

Implementation references

These project files define the behavior described above:

- `Assets/Script/Client/NPCProgressionManager.cs` - phases, movement, dialogue sequences, events, feature unlocks, and phase restoration.
- `Assets/Script/Client/NPCContractGiver.cs` - offer acceptance, reminder dialogue, and contract interactions.
- `Assets/Script/Client/ContractSO.cs` - contract dialogue, rewards, and build requirements.
- `Assets/Script/Scene/CinematicDirector.cs` - shots, playback rules, and cinematic events.
- `Assets/Script/Level/Tutorial/TutorialManager.cs` - tutorials and independent contract navigation guides.
- `Assets/Script/UI/Lessons/LessonTrigger.cs` - manually or interactively triggered lessons.
- `Assets/Script/Dialogue/DialogueTrigger.cs` - standalone dialogue and its completion event.

When planning a new action, first decide whether an existing event and component support it. If they do not, describe the desired start and completion conditions to the programming team before wiring the story.